# Операторы в C++

Арифметические операторы, логические операторы, приоритет операций

# Оператор и операнд

Оператором будем называть некоторый символ или набор символов, распознаваемый компилятором как некоторое действие, которое следует совершить над значениями (переменными или константами) ему переданными.

Входящие значения будем называть операндами.

В зависимости от количества входящих операндов разделим операторы на категории:

- унарные операторы – одно входящее значение;
- бинарные операторы – два входящих значения;
- тернарные операторы – три входящих значения.

Подавляющее большинство операторов в C++ являются бинарными. Тернарный оператор только один – условный оператор(?:).

# Арифметические операторы

# Арифметические операторы

Язык C++ позволяет записывать арифметические выражения с использованием операторов: +, -, * и /.

```
int a = 3, b = 5, c = 2;
int d = (a + b) * (a / c + b * c);
```

При вычислении арифметических выражений действуют правила приоритета: сначала выполняются операции умножения (*) и деления (/), затем операции сложения (+) и вычитания (-). Операторы с одинаковым приоритетом левоассоциативные (выполняются слева направо). С помощью круглых скобок, которые обладают более высоким приоритетом, чем арифметические операции можно задать свой порядок исполнения.

```
int a = 3, b = 5, c = 2;
int d = (a + b) * (a / c + b * c); // 8 * (1 + 10) // 88
int e = a + b * a / (c + b) * c;   // 3 + 5 * 2 / 7 * 2 // 5
```

# Арифметические операторы

Важно помнить, что при арифметических операциях над целыми числами результат так же остаётся целым числом.

```
int a = 3, b = 5, c = 2;
int d = a / b;  // 0
int e = b / c ; // 1
```

Операция деления (/) целых чисел возвращает целую часть от деления. Для вычисления остатка от деления можно использовать оператор %. При этом действует правило: c = a / b и d = a % b -> a = c * b + d.

```
int a = 3, b = 2;
int c = a / b; // 1
int d = a % b; // 1
int e = c * b + d; // 1 * 2 + 1 // 3
```

# Арифметические операторы

При использовании арифметических операторов происходит автоматическое привидение обоих операндо к одному типу. За это отвечают неявные преобразования типов.

1. При любой операции типы signed char и short расширяются до int, даже если мы складываем два числа типа short или два числа типа signed char.
2. Если один из типов знаковый, а второй беззнаковый, то результат тоже будет беззнаковым типом (unsigned).
3. Если один из типов меньше, чем другой, то расширение происходит до большего типа, не смотря на знак.
4. Если один из типов является типом с плавающей точкой, то результат тоже будет типом с плавающей точкой.

# Примеры неявных преобразований

```
short a = 3, b = 5;
auto x = a + b;
std::cout << x << " " << sizeof(x) << std::endl;
```

```
8 4
```

```
int i = 5;
unsigned j = 3;
auto x = j - i;
std::cout << x << std::endl;
```

```
4294967294
```

```
int i = 5;
unsigned long long j = 3;
auto x = j - i;
std::cout << x << " " << sizeof(x) << std::endl;
```

```
18446744073709551614 8
```

```
unsigned j = 3;
float f = 4.0f;
auto x = f / j;
std::cout << x << " " << sizeof(x) << std::endl;
```

```
1.33333 4
```

# Явные преобразования

В некоторых случаях требуется явно привести результат к определённому типу. В этом случае следует использовать явные преобразования типов. Выполнить явное преобразование можно с помощью конструкции static_cast или круглых скобок.

static_cast<имя типа, к которому приводим>(выражение, которое нужно преобразовать).

```
short a = 3, b = 5;
auto x = static_cast<short>(a + b);
std::cout << x << " " << sizeof(x) << std::endl;
```

```
8 2
```

```
int i = 3;
int j = 4;
auto x = (float)i / j;
std::cout << x << " " << sizeof(x) << std::endl;
```

```
0.75 4
```

# Присваивающие арифметические операторы

В некоторых ситуациях возникает необходимость изменить значение в некоторой переменной путём арифметической операции: увеличить вдвое, уменьшить на единицу, разделить на заданное число. Такие операции выглядят многословно:

```
int long_name_variable = 3;
long_name_variable = long_name_variable * 2;
```

Для сокращения записи часто применяются специальные операторы, которые берут значение из некоторой переменной, выполняют указанное действие и кладут результат обратно.

```
int long_name_variable = 3;
long_name_variable *= 2;
```

```
int long_name_variable = 3, a = 3, b = 2;
long_name_variable *= a + b;
```

Приоритет таких операторов ниже, чем у обычных арифметических операторов, что позволяет записывать сложные выражения.

# Инкремент и декремент

Особенно распространены выражения, в которых переменная уменьшается или увеличивается на единицу. Это касается разного рода счётчиков, индексаторов и т.п. По этой причине в C++ (и в C) есть специальные сокращённые операторы, которые выполняют увеличение и уменьшение на единицу.

инкремент

```
unsigned index = 0;
index = index + 1;
index += 1;
++index;
```

декремент

```
unsigned counter = 10;
counter = counter - 1;
counter -= 1;
--counter;
```

# Постфиксный инкремент / декремент

Если оператор инкремента стоит до имени переменной, то инкремент называется префиксным. Префиксный инкремент сразу изменяет значение переменной и возвращает изменённое значение. Иногда выгоднее сначала получить текущее значение, а затем увеличить значение на единицу. Если пользоваться только префиксным инкрементом, то это займёт две отдельные операции. Чтобы сократить запись можно воспользоваться постфиксным инкрементом – оператор инкремента, записанный после имени переменной.

```
int i = 3;
std::cout << ++i << '\n';
std::cout << i++ << '\n';
std::cout << i   << '\n';
```

```
4
4
5
```

# Постфиксный инкремент / декремент

При использовании постфиксного инкремента возникает отложенный побочный эффект: переменная изменяется не сразу, а в некоторый момент до начала вычисления следующего statement. Это может привести к выражениям, в которых результат может зависеть от порядка применения такого эффекта. Например:

```
int i = 1;
i = ++i + i++;
```

Следует ли прибавить единицу к i после того, как выполнится последнее присваивании или сначала применить побочный эффект, а затем произвести присваивание (что по сути отменит побочный эффект)?

В данной ситуации стандарт языка C++ не предписывает никаких определённых действий. Это называется неопределённым поведением (undefined behavior / UB). Результат такого кода может носить непредсказуемый характер.

# Логические операторы

# Логические операторы

В языке C++ есть логический тип bool, принимающий два возможных значения: true (истина) и false (ложь). Если в некотором случае нам нужно написать более сложное выражение, например такое, которое будет истинным только, если два значения сразу будут истинными, мы можем воспользоваться специальными логическими операторами:

Логическое И: && или and (бинарный оператор).
Логическое ИЛИ: || или or (бинарный оператор).
Логическое НЕ: ! или not (унарный оператор).

```
bool one = true, two = false;
std::cout << (one && two) << std::endl;
std::cout << (one || two) << std::endl;
std::cout << !one         << std::endl;
```

```
std::cout << (one and two) << std::endl;
std::cout << (one or two)  << std::endl;
std::cout << (not one)     << std::endl;
```

```
false
true
false
```

Логические операторы обладают более низким приоритетом, чем операторы вывода.

# Операторы отношений

# Операторы отношений

Язык C++ позволяет проводить сравнение чисел и сравнивать их. Для этого используются операторы отношений следования и равенства:

- Меньше <: возвращает true, если левый операнд идёт раньше правого, иначе – false.
- Больше >: возвращает true, если левый операнд идёт после правого, иначе – false.
- Равенство ==: возвращает true, если оба операнда одинаковы, иначе - false.
- Меньше или равно <=: отрицание к оператору >.
- Больше или равно >=: отрицание к оператору <.
- Неравно !=:  отрицание к оператору ==.

```
int a = 3, b = 5, c = 5;

std::cout << (a < b) << std::endl;
std::cout << (a > b) << std::endl;
std::cout << !(b < c) << " " << (b >= c) << std::endl;
std::cout << !(b > c) << " " << (b <= c) << std::endl;
std::cout << (b == c) << " " << (a == b) << std::endl;
std::cout << !(b == c) << " " << (b != c) << std::endl;
```

```
true
false
true true
true true
true false
false false
```

Операторы сравнения также обладают более низким приоритетом, чем операторы вывода, но более высоким, чем логические операторы.

# Операторы сравнения

Дано кольцо с центром в начале координат внешнего радиуса $R$ и внутреннего радиуса $r$ ($r < R$). Для пары целочисленных координат $(x, y)$ определить, попадает ли точка в кольцо.

```
3   #include <iostream>
4   int main() {
5       std::cout << std::boolalpha;
6       unsigned r, R;
7       std::cin >> r >> R;
8       int x, y;
9       std::cin >> x >> y;
10      auto d_sqr = x * x + y * y;
11      std::cout << (d_sqr > r * r and d_sqr < R * R) << std::endl;
12
13      return 0;
14  }
```

# Побитовые (поразрядные) операторы

# Побитовая (поразрядная) логика

Напомним, что информация в памяти хранится в виде последовательностей нулей и единиц. В частности, беззнаковые целые числа хранятся ,как представления чисел в двоичной системе исчисления.

Каждый бит можно воспринимать как правду или ложь в зависимости от значения 1 или 0, соответственно. Тогда над отдельным битом можно производить операции, которые соответствуют логическим, причём такие операции можно одновременно произвести над всеми битами числа. Такие операторы называются побитовыми логическими операторами. Они применяются к целочисленным значениям.

- & - побитовое И, бинарный оператор, выполняет конъюнкцию над каждой парой битов в одинаковых разрядах;
- | - побитовое ИЛИ, бинарный, выполняет дизъюнкцию над каждой парой битов в одинаковых разрядах;
- ^ - побитовое «Исключающее ИЛИ», выполняет операцию сложения по модулю 2 над каждой парой битов в одинаковых разрядов;
- ~ - побитовое НЕ, унарный оператор, инвертирует каждый бит.

# Операторы сдвига

Оператор << - бинарный оператор производящий сдвиг влево битов, свободные биты занимают нули, ушедшие биты отбрасываются, несмотря на знак числа.

Оператор >> - бинарный оператор производящий сдвиг вправо битов, свободные биты заполняются нулями для беззнаковых чисел или знаковым битом для чисел со знаком, ушедшие биты отбрасываются.

Операторы сдвига имеют такой же приоритет, что и операторы вывода. По существу, операторы вывода являются *перегруженными* операторами сдвига, т.е. имеют такую же запись, но иной смысл при использовании с объектами std::cout или std::cin.

# Таблица приоритета

| Выше положение в таблице – выше приоритет | | |
|---|---|---|
| `() [] . ->` | Операторы обращения | |
| `+ - ~ ! * ++ -- sizeof` | Унарные операторы | Справа налево |
| `* / %` | Мультипликативная арифметика | |
| `+ -` | Аддитивная арифметика | |
| `>> <<` | Операторы сдвига | |
| `< > <= >=` | Отношение порядка | |
| `== !=` | Отношение равенства | |
| `&` | Побитовый AND | |
| `^` | Побитовый XOR | |
| `\|` | Побитовый OR | |
| `&&` | Логический AND | |
| `\|\|` | Логический OR | |
| `?:` | Тернарный условный оператор | Справа налево |
| `= *= /= %= += -= <<= >>= &= \|= ^=` | Арифметика с присваиванием | Справа налево |

# Оператор последовательного исполнения

# Оператор последовательного исполнения

Оператор , (запятая) - бинарный оператор, вычисляет левый операнд и отбрасывает его, затем вычисляет правый операнд и возвращает результат.

```
int main() {
    int i = 5, j = 7, k = 9;
    auto x = (++i, ++j);
    std::cout
        << "x = " << x << '\n'
        << "i = " << i << '\n'
        << "j = " << j << '\n'
    << std::endl;

    return 0;
}
```

```
x = 8
i = 6
j = 8
```

Оператор последовательного исполнения имеет самый низкий приоритет.